Guía para principiantes · Atacar una API vulnerable usando Postman

> Esta guía es para alguien que **empieza en tecnología**. No necesitas saber programar. Vas a ponerte en el papel de un atacante y descubrir, con ejemplos muy sencillos, **los 12 fallos de seguridad** más típicos de una API. En cada paso te explico **qué haces y por qué funciona el ataque**.
> Usaremos la **VulnBank API** en su versión **vulnerable** (la insegura, hecha así a propósito). El banco es inventado y el dinero es ficticio: **puedes atacar con total tranquilidad**, no rompes nada real.

Antes de empezar, lee esto

- Esto es un **entorno de entrenamiento**. Solo puedes atacar **esta** API de práctica (`/api/lab/…`). Atacar APIs o webs reales sin permiso es **ilegal**.
- Todos los "secretos" que verás (claves, contraseñas) son **inventados** y no sirven para nada fuera del laboratorio.
- Si algo se desordena, siempre puedes **dejar todo como al principio** (ver el final de la guía).

Primero, 4 ideas básicas

1. ¿Qué es una API? Piensa en un restaurante. Tú (el cliente) no entras a la cocina: le pides las cosas al **camarero**, y él te trae lo que pediste. Una **API** es ese camarero: un programa al que le pides datos ("¿cuánto dinero tengo?") y te responde.

2. ¿Qué es Postman? Es un programa gratuito para **hacerle pedidos a una API** sin escribir comandos: rellenas casillas y pulsas un botón. Es como un formulario para hablar con el camarero.

3. ¿Qué es un token? Es como la **pulsera de un concierto**: cuando demuestras quién eres (con usuario y contraseña) te dan una pulsera. Luego, en cada pedido, enseñas la pulsera.

4. ¿Qué es una vulnerabilidad? Es un **descuido** en cómo está hecha la API que permite hacer algo que no deberías: ver datos de otra persona, saltarte una contraseña, etc. Una API vulnerable **confía** en lo que le pides sin comprobarlo. Eso es justo lo que vamos a aprovechar.

Conoce la pantalla de Postman

Cuando abres un pedido nuevo verás estas zonas (de arriba abajo):

Zona	Para qué sirve
Método (desplegable, dice `GET`)	El tipo de pedido: `GET` = **pedir** datos, `POST` = **enviar** datos, `PATCH` = **cambiar** datos.
Barra de dirección	A qué dirección le haces el pedido.
Botón azul **Send**	Envía el pedido.
Pestaña **Headers**	Información extra del pedido (aquí irá la pulsera).
Pestaña **Body**	Los datos que envías (usuario, contraseña, importe…).
Panel **Response** (abajo)	La respuesta de la API y su **código**: `200` = salió; `401` = sin identificar; `404` = no existe.

Paso 1 · Instalar Postman y preparar la libreta

1. Entra en **postman.com/downloads**, instala la **aplicación de escritorio** y crea una cuenta gratuita.
2. Crea una libreta de datos (un **Environment**): barra izquierda → **Environments** → **+**. Ponle de nombre `VulnBank` y añade dos filas:

Variable	Valor
`api`	`https://yellow-ground-024d78310.6.azurestaticapps.net/api/lab`
`token`	*(déjalo vacío)*

3. Guarda (**Cmd + S** / **Ctrl + S**) y, **arriba a la derecha**, en el desplegable que dice *No environment*, elige **VulnBank**.

A partir de ahora, escribir `{{api}}` equivale a la dirección completa.

Paso 2 · Comprobar la versión y entrar como Laura

Comprobar que la versión es la vulnerable. Pedido nuevo (**+**), método **GET**, dirección `{{api}}/version`, pulsa **Send**. Debe decir `"version": "vulnerable"`. Si dice `"endurecida"`, pide a Claude *"coloca el laboratorio de API vulnerable"*.

Entrar como Laura. Pedido nuevo, método **POST**, dirección `{{api}}/auth/login`, pestaña **Body → raw → JSON**, y pega:

```
{
  "username": "laura",
  "password": "Laura2026!"
}
```

Pulsa **Send**. En la respuesta, copia el texto largo del `token` (empieza por `eyJ`) y guárdalo en la libreta: **Environments → VulnBank → fila `token`**, pega y guarda.

> A partir de aquí, los pedidos privados llevan una **cabecera**. En la pestaña **Headers** añade una fila: **Key** `X-Auth-Token`, **Value** `{{token}}`. Es tu pulsera. (Cuando un ataque **no** la necesite, lo digo.)

Para situarte, mira tu cuenta (la de Laura es la **5001**): método **GET**, dirección `{{api}}/accounts/5001`, con la cabecera puesta. Verás **2450,75 €**.

Los 12 ataques

En cada uno te digo qué poner en Postman. Donde no se mencione **Headers** o **Body**, es que ese pedido no los necesita.

Ataque 1 · Ver la cuenta de otra persona

La idea: la dirección termina en el número de la cuenta. Si cambio mi número (5001) por el de otra persona (5002), una API segura me lo impediría. Esta no.

- **GET** · `{{api}}/accounts/5002` · Headers: `X-Auth-Token` = `{{token}}`

Qué consigues: ves el saldo y los datos de **Carlos**, que no es tuyo. Este fallo se llama **BOLA**.

Ataque 2 · Colarte como administrador con una "pulsera falsa"

La idea: la pulsera lleva una firma que debería impedir falsificarla, pero la API acepta pulseras que dicen "no tengo firma". Usa esta pulsera ya preparada que dice "soy el usuario 1003" (el admin). **Cambia solo la cabecera**:

- **GET** · `{{api}}/users/me` · Headers: `X-Auth-Token` = `eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdWIiOjEwMDN9.`

Qué consigues: la API te saluda como **admin** sin su contraseña. (Al terminar, vuelve a poner `{{token}}` en la cabecera.)

Ataque 3 · Adivinar contraseñas

Parte A (ver quién existe). Haz dos logins (como en el paso 2) cambiando el Body, y compara el mensaje:

- `{"username":"pepito","password":"x"}` → dice *"El usuario no existe"*.
- `{"username":"carlos","password":"x"}` → dice *"Contraseña incorrecta"*.

Ya sabes que `carlos` existe.

Parte B (probar contraseñas). Repite el login de Carlos cambiando la contraseña por `password`, `123456`, `qwerty`… Verás `401` en casi todas, pero `123456` **responde** `200`: esa es su contraseña.

> La API no bloquea los intentos. En Postman puedes repetirlo a mano; con un archivo de contraseñas, el **Runner** de Postman lo haría solo. Lo importante es la idea: nada te frena.

Ataque 4 · Ver datos privados de más

La idea: pides tu perfil y la API devuelve más de lo que debería.

- **GET** · `{{api}}/users/me` · Headers: `X-Auth-Token` = `{{token}}`

Qué consigues: además del nombre y el email, aparecen el **DNI**, el **PIN** y la **contraseña cifrada**.

Ataque 5 · Ascenderte a administrador tú mismo

La idea: el pedido para editar tu perfil acepta cualquier campo, incluido `role`.

- **PATCH** · `{{api}}/users/me` · Headers: `X-Auth-Token` = `{{token}}`
- Body → raw → JSON:

```
{ "role": "admin" }
```

Qué consigues: tu perfil pasa a `"role": "admin"`.

Ataque 6 · Entrar al panel de administración siendo cliente

La idea: el panel comprueba que tengas pulsera, pero no que seas admin.

- **GET** · `{{api}}/admin/users` · Headers: `X-Auth-Token` = `{{token}}`

Qué consigues: la lista de **todos** los usuarios y sus datos.

> Si hiciste el ataque 5, restaura los datos (ver el final) y vuelve a iniciar sesión para una pulsera nueva.

Ataque 7 · Robar dinero con una transferencia "negativa"

La idea: si envías una cantidad **negativa**, el dinero va al revés.

- **POST** · `{{api}}/transfers` · Headers: `X-Auth-Token` = `{{token}}`
- Body → raw → JSON:

```
{ "fromAccountId": 5001, "toAccountId": 5002, "amount": -1000 }
```

Qué consigues: tu saldo **sube** 1000 € y el de Carlos baja.

Ataque 8 · Hacer que el servidor espíe por ti (SSRF)

La idea: la "vista previa de un enlace" hace que **el servidor** visite la dirección que tú le des, incluso sitios internos.

- **GET** · `{{api}}/tools/preview?url=http://169.254.169.254/latest/meta-data/` · Headers: `X-Auth-Token` = `{{token}}`

Qué consigues: unas "claves de la nube" (ficticias; el ataque está **simulado** en el laboratorio).

Ataque 9 · Encontrar una puerta de servicio olvidada

La idea: quedó abierta una dirección de "pruebas" que muestra secretos. **No necesita pulsera.**

- **GET** · `{{api}}/debug`

Qué consigues: la **contraseña secreta** con la que la API firma las pulseras y la dirección de su base de datos.

Ataque 10 · Usar una versión antigua de la API

La idea: la API vieja (`v1`) sigue encendida y no pide identificarse. **No uses pulsera** (quita o desmarca la cabecera):

- **GET** · `{{api}}/v1/users/1002`

Qué consigues: todos los datos de Carlos sin entrar a ninguna cuenta.

Ataque 11 · Entrar sin contraseña con un "truco mágico"

La idea: en vez de una contraseña de texto, envías `{"$ne": ""}`, que significa *"cualquier cosa distinta de vacío"*. La API lo toma como *"la contraseña vale siempre"*.

- **POST** · `{{api}}/auth/login` · Body → raw → JSON:

```
{ "username": "admin", "password": { "$ne": "" } }
```

Qué consigues: una pulsera de **administrador** sin su contraseña. (Es una *inyección*.)

Ataque 12 · Tumbar la API pidiéndole demasiado

La idea: al pedir movimientos, tú decides cuántos. Pide una cantidad enorme y fíjate en el tamaño de la respuesta (abajo a la derecha, junto al tiempo):

- **GET** · `{{api}}/accounts/5001/transactions?limit=5` → respuesta pequeña.
- **GET** · `{{api}}/accounts/5001/transactions?limit=20000` → respuesta enorme.

Qué consigues: la segunda es miles de veces más grande. Repetida muchas veces, dejaría la API de rodillas (*denegación de servicio*). **No** la lances en bucle contra el sitio.

La gran lección

Todos estos ataques tienen **la misma raíz**: la API **se fía** de lo que le manda el cliente. Una API segura, como la versión endurecida de este mismo laboratorio, hace lo contrario: comprueba **quién eres**, **si lo que pides es tuyo**, **si tienes permiso** y **si los datos tienen sentido**. Para ver esos 12 ataques fallando, pide a Claude la versión endurecida y usa su guía de Postman.

Para terminar · Dejar todo como al principio

- **POST** · `{{api}}/reset` (sin Body ni Headers)

Devuelve saldos, perfiles y roles a sus valores iniciales.

Si algo sale mal

Ves esto	Qué pasa	Qué hacer
`"version": "endurecida"`	Está la versión segura: los ataques no funcionan.	Pide *"coloca el laboratorio de API vulnerable"*.
`{{api}}` en rojo	No elegiste el environment VulnBank.	Elígelo arriba a la derecha (paso 1).
`401` · `Falta un token válido`	La pulsera caducó (15 min) o falta la cabecera.	Inicia sesión otra vez y revisa la pestaña **Headers**.
Un ataque devuelve `404` donde esperabas datos	Pusiste pulsera donde el ataque dice que **no** lleva (ataques 9 y 10), o al revés.	Marca/desmarca la casilla de la cabecera `X-Auth-Token`.
Los saldos quedaron raros	Hiciste transferencias de prueba.	Haz el pedido de "Para terminar".

Mini-diccionario

Palabra	Qué significa
API	Programa que recibe pedidos y devuelve datos (el camarero).
Postman	Programa para hacerle pedidos a una API rellenando casillas.
GET / POST / PATCH	Pedir / enviar / cambiar datos.
Token / pulsera	Lo que demuestra quién eres en cada pedido. Caduca.
Header (cabecera)	Información extra del pedido, como la pulsera.
Body (cuerpo)	Los datos que envías en un pedido.
Vulnerabilidad	Un descuido que permite hacer algo que no deberías.
BOLA	Ver objetos de otras personas solo cambiando su número.
Inyección	Colar un "truco" en los datos para engañar a la API.
SSRF	Lograr que el servidor visite sitios por ti.

Anexo · Colección lista para importar (opcional)

Para no crear los pedidos a mano existe el archivo `VulnBank-API-vulnerable.postman_collection.json`, con los 12 ataques ya preparados y numerados como en esta guía.

1. En Postman pulsa **Import** y arrastra el archivo.
2. Aparece la colección **VulnBank API (vulnerable · ATAQUES)**.
3. Ejecuta primero **"0 · Iniciar sesión (Laura)"**: el token se guarda solo.
4. Lanza los ataques en orden. Cada uno comprueba que el ataque **funcionó** (por ejemplo, que el ataque 1 devuelve la cuenta de Carlos).

> La colección ya trae la dirección de la API; con el environment **VulnBank** elegido se usan sus valores. Los ataques 9 y 10 van sin pulsera a propósito.